ExSpike: A General Full-Event Neuromorphic Architecture for Exploiting Irregular Sparsity with Event Compression

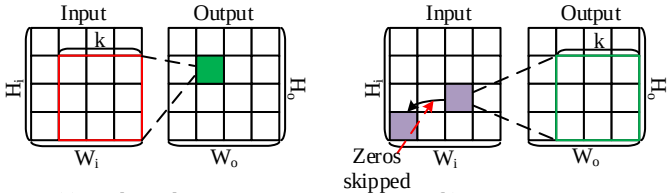
Yuehai Chen and Farhad Merchant
Bernoulli Institute and CogniGron, University of Groningen, The Netherlands
Email: {yuehai.chen, f.a.merchant}@rug.nl

***Abstract*—Spiking neural networks (SNNs) promise energy-efficient computing due to their sparse spatio-temporal activity. However, effectively translating such irregular sparsity into practical performance and energy gains remains challenging, as full-event computing architectures are still underexplored. This paper proposes ExSpike, a general full-event neuromorphic architecture that fully exploits irregular sparsity in SNNs. To realize pure event-driven execution, we first propose a set of dataflow optimizations to ensure that the inputs to each SNN layer remain spike-based, thereby enabling full-event execution throughout the network. We then design a hardware-efficient full-event architecture, named ExSpike, which supports the optimized pure event-driven dataflow and an additional Attention Core for spike-driven self-attention. To further improve computing efficiency, we introduce adjacent-position event compression to reduce redundant accumulations across spatially adjacent spike sequences. ExSpike is implemented on an AMD Xilinx Virtex-7 FPGA and evaluated on both classification and segmentation workloads. Experimental results show that ExSpike achieves high normalized energy efficiency across diverse SNN models while maintaining competitive accuracy, delivering up to 479.15 GOPS, 281.85 GOPS/W, and 0.80 GOPS/W/PE. In particular, ExSpike achieves up to 10× higher normalized energy efficiency than prior FPGA-based accelerators.**

***Index Terms*—Spiking neural network, event convolution, zero-aware, hardware architecture, spiking transformer**

I. Introduction

Spiking neural networks (SNNs) were proposed in the 1990s and are regarded as the third generation of neural networks [1]. In recent years, SNNs have gained significant attention because of their closer alignment with biological mechanisms and an efficient event-driven computation model. Recent advances in training algorithms and applications [2]–[5] have significantly improved the capability of SNNs. In particular, recent object detectors based on spiking convolutional neural networks (SCNNs) have achieved 67.2% mAP@50 on Gen1 [6], which is 2.5% higher than an artificial neural network (ANN) with an equivalent architecture, while improving energy efficiency by 5.7×. Furthermore, the development of spike-driven self-attention (SDSA) has enabled spiking transformers [7]–[9] to emerge as a promising architecture for complex workloads. These advances, together with the inherent sparsity of SNNs, provide substantial opportunities for improving computational efficiency. However, modern general-purpose processors such as central processing units (CPUs) and graphics processing units (GPUs) are not well suited to the sparse, irregular computation and memory access patterns of SNNs, and thus cannot fully exploit their event-driven nature. This mismatch has motivated extensive research on specialized SNN hardware, including digital, analog, and mixed-signal neuromorphic processors [10]–[12]. As shown in Fig. 1, the dataflow in SCNNs can be divided into two fundamental types that differ in how convolution is triggered and propagated. Fig. 1(a) shows the traditional convolution (TConv) dataflow, in which each output neuron corresponds to a receptive field and requires weighted accumulation over all input channels within that region. In contrast, Fig. 1(b) illustrates the event-driven convolution (EConv) dataflow, where each input spike event has a fixed spatial influence range and simultaneously affects neurons at the same spatial location across all output channels. As a result, every operation directly contributes to valid updates, avoiding the workload imbalance introduced by neuron-level computation.



(a) Traditional conv. (b) Event conv.

$$MAC_{traditional\_conv} = H_o \cdot W_o \cdot C_o \cdot C_i \cdot k^2$$
$$MAC_{event\_conv} = \underbrace{\alpha \cdot C_i \cdot H_i \cdot W_i}_{N_{event}} \cdot C_o \cdot k^2$$
$$MAC_{traditional\_conv}/MAC_{event\_conv} = (H_o \cdot W_o) / (\alpha \cdot H_i \cdot W_i)$$

C_i: input channels
C_o: output channels
α: active ratio

(c) Analysis of calculation.

Fig. 1: The computation model of different convolutions.

Based on this distinction, Fig. 1(c) compares the computation cost of the two schemes. TConv incurs a fixed cost determined by the spatial dimensions and channel counts, whereas the cost of EConv scales with the number of active events and therefore benefits significantly from input sparsity. Fig. 2 further reports the layer-wise latency of TConv and EConv in VGG11 [13], along with the corresponding input spike sparsity. EConv outperforms TConv in every layer, achieving up to 97% latency reduction at Conv-7 and 88%

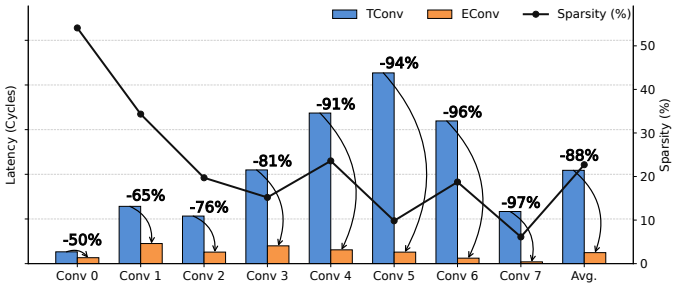


Fig. 2: Layer-wise VGG11 on CIFAR-10 performance comparison: TConv vs. EConv.

reduction on average. The results also show that higher input sparsity leads to larger speedups. For example, when only 10% of spikes are effective, corresponding to 90% sparsity, the runtime can ideally be reduced by 90%. Therefore, given the inherently low activity of SNNs, EConv can substantially reduce redundant accumulations and memory traffic while preserving correctness, thereby improving throughput and reducing energy consumption.

Although several neuromorphic processors for SNNs support sparsity-aware processing and direct-encoding acceleration [14]–[16], three important challenges remain. First, direct coding requires multi-bit multiply-accumulate (MAC) operations, and average pooling generates floating-point intermediate results, both of which interrupt pure event-driven execution. Second, traditional convolution mapping is often inefficient and prone to workload imbalance, since each processing element (PE) is typically assigned to one output neuron; under irregular sparsity, this results in low PE utilization and increased latency. Third, full-event neuromorphic architectures are still underexplored, limiting the ability of existing designs to fully exploit the inherent sparsity of SNNs.

To address these challenges, we propose ExSpike, a general full-event neuromorphic architecture. By jointly optimizing dataflow and architecture, ExSpike reduces these bottlenecks and improves both energy efficiency and end-to-end performance. The main contributions are as follows.

1) ***Dataflow:*** We identify the key operations in SNN models that interrupt pure event-driven execution, and propose a set of optimizations to restore a fully event-driven dataflow, including direct coding, event-driven convolution, and event-driven average-pooling/fully connected computation.
2) ***Architecture:*** We propose ExSpike, a hardware-efficient full-event neuromorphic architecture that matches the optimized pure event-driven dataflow and integrates an Attention Core to support SDSA computation. In addition, we introduce adjacent-position event compression (APEC) to merge redundant events across adjacent spatial positions, thereby further reducing computation cost.
3) ***Implementation:*** We implement ExSpike on an AMD Xilinx FPGA and evaluate it on diverse SNN workloads, including VGG11, ResNet18, spiking transformer models, and image segmentation networks. Experimental results show that ExSpike achieves up to 479.15 GOPS, 281.85 GOPS/W, and 0.80 GOPS/W/PE, while maintaining competitive accuracy across the evaluated workloads. These results demonstrate the efficiency of ExSpike and its support for diverse SNN models and operators.

II. Pure Event-Driven Dataflow Optimization

Although the basic data representation in SNNs is spike-based, some operators or processing stages may generate non-spike intermediate results, which are then forwarded to subsequent layers and break the event-driven execution flow. Typical examples include the direct coding scheme and the average-pooling operator. In this section, we introduce a set of optimizations that enable the pure event-driven execution of SNN models.

A. Pure Event-Driven Convolutional Computing Dataflow

Optimization I: Direct Coding. The direct coding layer at the network input poses a major challenge to pure event-driven execution, because its input activations are multi-bit fixed-point values rather than binary spikes. For this reason, some previous work executed the coding layer on the host side and transferred only the generated spike sequences to the accelerator [17]–[19]. To avoid such off-chip preprocessing, we exploit the shift-and-accumulate formulation of binary multiplication to support direct coding within the accelerator. Specifically, as summarized under **Optimization I (OPT1)** in Algorithm 1, the input activations are quantized into signed fixed-point values and then bit-sliced, while the weights are duplicated and shifted accordingly to match the bitwise multiplication process in the coding layer.

Optimization II: Event-Driven Convolution. Unlike conventional convolution accelerators, where each PE is typically mapped to a neuron for kernel computation and then time-multiplexed to support all output neurons, the irregular sparsity of different kernels can lead to severe workload imbalance across PEs, while time multiplexing further increases latency. In contrast, in event-driven convolution, each valid spike event corresponds to a well-defined target update region. By partitioning the PE array into multiple blocks, each block can be dedicated to computing one target kernel region, so that every active cycle contributes directly to neural inference. The proposed event-driven convolutional dataflow is summarized under **Optimization II (OPT2)** in Algorithm 1. For each convolutional layer, spike events are first collected at the same spatial location across input channels (line 9). If valid events exist, weight accumulation is performed according to their positions (lines 10–11). Furthermore, since a single spike event affects the same receptive-field region across all output channels, channel-level parallelism can be exploited to reduce latency while maintaining high PE utilization. When the number of output channels exceeds the available hardware parallelism, the architecture is reused across multiple groups to complete the updates for all output neurons (lines 5–6).

B. Pure Event-Driven Fully Connected Computing Dataflow

Optimization III: Event-Driven Average-Pooling and Fully Connected Computation. In some SNN models, an average-

Algorithm 1 Optimizations for Pure Full-Event Computation

Require: Input map $\mathbf{S}$ of an SNN layer; data precision B; convolution weight $\mathbf{W}$; FC weight $\mathbf{W}_{fc}$; output channels C_o; input size $H_i \times W_i$; architecture parallelism P

Optimization I: Direct Coding

if first_layer **then**
  $\mathbf{S} \leftarrow \mathbf{S}$.QUANTIZE($B$).BITSLICE($B$)
  $\mathbf{W} \leftarrow \mathbf{W}$.DUPLICATESHIFT($B$)
end if

Optimization II: Event-Driven Convolution

$G \leftarrow \lceil C_o/P \rceil$
for g = 1 to G **do**
  for h = 1 to H_i **do**
    for w = 1 to W_i **do**
      for all valid event e in $\mathbf{S}[:, h, w]$ **do**
        $\mathbf{psum}_g \leftarrow \mathbf{W}_g$.CAL($e$)
        $\mathbf{MP}_g \leftarrow \mathbf{MP}_g + \mathbf{psum}_g$
      end for
    end for
    $\mathbf{MP}_g$.CHECKANDFIRE()
  end for
end for

Optimization III: Event-Driven Average-Pooling and Fully Connected Computation

$\mathbf{W}_{fc} \leftarrow \mathbf{W}_{fc}/pooling\_size^2$
for h = 1 to H_i **do**
  for w = 1 to W_i **do**
    for all valid event e in $\mathbf{S}[:, h, w]$ **do**
      $\mathbf{y}_{fc} \leftarrow \mathbf{y}_{fc} + \mathbf{W}_{fc}$.CAL($e$)
    end for
  end for
end for

pooling layer is inserted between the last convolutional layer and the first fully connected (FC) layer to preserve globally aggregated features and reduce the computational complexity of the FC layer. However, average pooling makes end-to-end event-driven computation difficult because it produces floating-point intermediate results. NEURAL [20] proposed a window-to-time-to-first-spike (W2TTFS) mechanism to avoid the floating-point computation introduced by average pooling. However, this approach is not fully event-driven, since it still requires additional counters to record the number of valid events. In this work, we propose an event-driven average-pooling fully connected (EAFC) dataflow. As summarized under **Optimization III (OPT3)** in Algorithm 1, we first scale the FC weight matrix according to the pooling size. Then, similar to event-driven convolution, we check the valid events at each spatial position and perform event-driven updates of the FC neurons $\mathbf{y}_{fc}$.

With these three optimizations, the SNN model can be executed in a pure event-driven manner without using multipliers. In the next section, we present the proposed full-event hardware architecture that supports these optimizations.

III. Architecture Design and Optimization

Fig. 3 illustrates the overall architecture of ExSpike, which consists of four computing cores: the Sparse Core, the event-driven processing element (EPE) Core, the Attention Core, and the event-driven average-pooling and fully connected (EAFC) Core. In the Sparse Core, spike sequences are fetched from either the Spike SRAM or the Residual Spike SRAM, depending on the source spikes of the current SNN layer. Valid event addresses are then extracted by the fast event filter and stored in the address-event representation (AER) FIFO. When the AER FIFO is non-empty, it triggers the computation of the EPE Core.

The EPE Core serves as the primary execution engine to support the optimized dataflow ***(OPT2)*** and comprises 32 EPE clusters. Each EPE cluster contains a weight processing element (WPE) block, a membrane potential processing element (MPE), and a fire processing element (FPE). The WPE block performs weight accumulation and writes the results to the elastic FIFO (eFIFO). When the eFIFO is non-empty, the MPE updates the membrane potentials of neurons. The FPE then performs bias accumulation and threshold comparison to generate output spikes. In this way, each EPE cluster updates the membrane potential of one output channel, allowing the EPE Core to process 32 output channels in parallel and thereby improve computational throughput. Moreover, the EPE Core also supports ***OPT1***, where weight duplication and shifting are performed implicitly during reads from the Weight SRAM.

The Attention Core is responsible for spike-driven self-attention computation. It receives output spikes from the EPE Core and performs attention computation when the attention mechanism is enabled; otherwise, it directly writes the spikes back to the Spike Buffer. The EAFC Core supports the fused computation of average pooling and fully connected operations ***(OPT3)***, thereby enabling a full-event computing flow.

Finally, ExSpike can flexibly support different SNN topologies by programming the Instruction SRAM with model parameters, such as kernel size and channel count. The fetcher and decoder then read and decode the instructions to perform layer-by-layer inference.

A. Event-driven Convolution Hardware Architecture

1) Proposed Architecture Features

Fig. 4 illustrates the hardware implementation of the proposed event-driven convolution dataflow, which consists of five main components. First, the spike sequence storage strategy ❶ is adopted, in which each address in the Spike SRAM stores spike data from all input channels at the same spatial location. Next, the fast event filter ❷ identifies one valid event position per cycle. Specifically, it first extracts the one-hot code corresponding to the lowest active bit and then uses this code as an index to obtain the corresponding valid position from a look-up table. The resulting event position is then written into the AER FIFO ❸. When the AER FIFO is non-empty, indicating the presence of valid events, the EPE Core fetches an event index from the FIFO, reads the corresponding weight matrix from the Weight SRAM, and

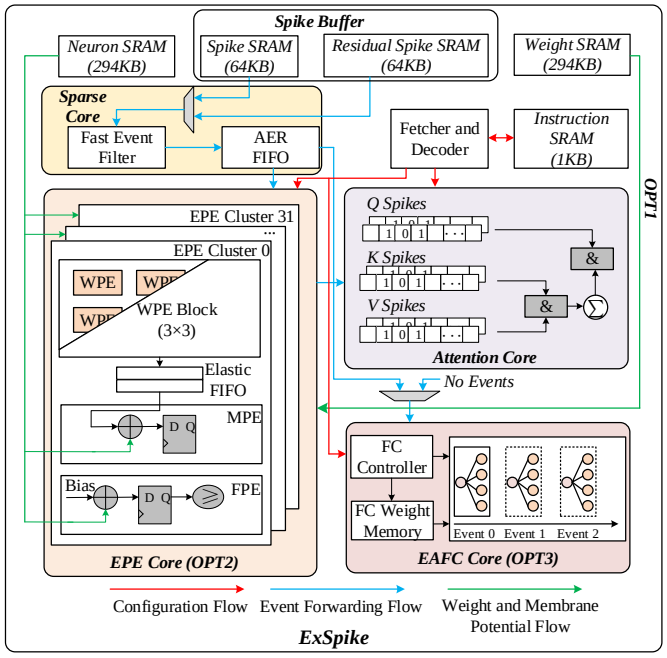


Fig. 3: The overall architecture of ExSpike.

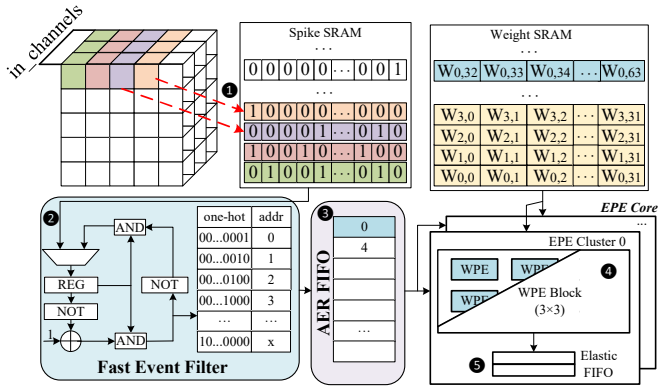


Fig. 4: The hardware implementation of event-driven convolution computing.

performs weight accumulation. In the proposed architecture, the EPE Core contains 32 parallel EPE Clusters, enabling the accumulation of convolution weights for 32 output channels associated with the same target region in parallel. Accordingly, the Weight SRAM stores the convolution weights of different input channels across these 32 output channels. Each WPE block further integrates 3×3 WPE units to compute the weights of a single convolution kernel ❹. Once the AER FIFO becomes empty and the fast event filter is idle, both event filtering and event computation are complete. At this point, the partial sums generated by the EPE clusters are written into the elastic FIFO ❺, which serves as the input source for the subsequent MPE stage.

2) Adjacent-position Event Compression (APEC)

To further improve event computing efficiency, we propose a novel event compression mechanism, namely adjacent-position event compression (APEC). Input spike sequences

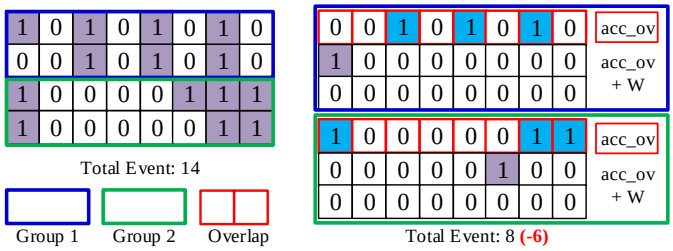


Fig. 5: The implementation of adjacent position event compression.

are grouped by spatial adjacency. As illustrated in Fig. 5, adjacent spatial positions are partitioned into groups, and each group is compressed by extracting the shared overlap among its spike sequences. For each group, we first apply bit-wise AND logic to identify the overlap sequence, and then derive the corresponding non-overlap sequences that are disjoint from the overlap. During computation, the overlap sequence is processed first and its partial sums are cached, after which the non-overlap sequences are updated using only their unique event contributions. In this way, repeated accumulations caused by overlapped events can be avoided, thereby reducing both memory traffic and computation cost.

Fig. 5 provides a concrete example. After compression, the total number of events is reduced from 14 to 8. For a 3×3 convolution with 64 output channels, this directly eliminates $6 \times 64 \times 9 = 3456$ accumulation operations. Since APEC only reorganizes the execution order of overlapped and non-overlapped events, it preserves numerical equivalence with the original full-event execution. For theoretical analysis, we consider a single APEC group of size g. APEC is motivated by the observation that adjacent spatial positions often exhibit correlated spike activities, which leads to a non-negligible overlap among their spike sequences. Let $S_i \subseteq \{1, 2, \ldots, C_i\}$ denote the active-channel set of the i-th spike sequence, where C_i is the number of input channels. The common overlap of the group is defined as

$$O_G = \bigcap_{i=1}^{g} S_i. \tag{1}$$

Let $\Phi(c)$ denote the convolution contribution of an input event c. Since each spike sequence can be decomposed into the shared overlap and its non-overlap part, APEC computes the overlap only once and then adds the remaining unique contributions of each sequence individually. Therefore, APEC is numerically equivalent to the original full-event execution.

Accordingly, the number of eliminated redundant events is

$$\Delta N_{\text{event}}^{(g)} = (g-1)|O_G|. \tag{2}$$

Since each valid input spike event affects all output channels over all k^2 kernel positions, the corresponding accumulation reduction brought by APEC can be expressed as

$$\Delta C^{(g)} = \Delta N_{\text{event}}^{(g)} C_o k^2 = (g-1)|O_G| C_o k^2. \tag{3}$$

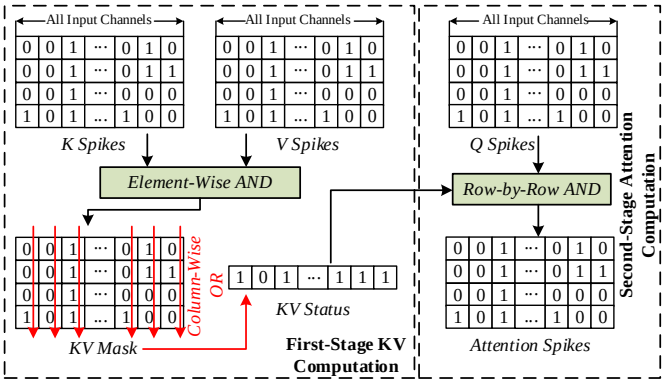


Fig. 6: The implementation of attention core.

This equation shows that the gain of APEC is jointly determined by the group size and the overlap size, where $|O_G|$ directly reflects the correlation among adjacent spike sequences.

The main overhead of APEC lies in the storage and reuse of overlap partial sums, which can be approximated as

$$M_{\text{ov}} \approx C_o k^2 w_{\text{acc}}, \tag{4}$$

where w_{acc} denotes the bitwidth of the partial sum. Therefore, although a larger group provides a higher theoretical reuse factor, the overall gain of APEC does not necessarily increase monotonically with group size because the higher-order overlap $|O_G|$ typically decreases as more spike sequences are included. This implies the existence of an optimal group size, as discussed in Section IV-A.

B. Implementation of EAFC Core

As shown in Fig. 3, when an average-pooling layer is followed by a fully connected (FC) layer, the EAFC Core is activated to process valid events forwarded from the Sparse Core. Specifically, upon receiving a valid event, the FC Controller generates the corresponding read address for the FC Weight Memory and triggers FC computation. Therefore, the FC computation is carried out in a fully event-driven manner. To fuse average pooling with the FC layer, the FC weights are scaled offline during the generation of ExSpike configuration files. For example, for a pooling window of size 4×4, each FC weight is divided by 16 before the FC weight configuration files are generated.

C. Implementation of Attention Core

Fig. 6 illustrates the hardware implementation of the Attention Core. Since attention-spike generation involves the processing of Q, K, and V spike sequences, the computation is divided into two stages. In the first stage, ExSpike performs KV computation. The K spikes are generated first, followed by the V spikes. An element-wise AND operation between the K and V spikes is then used to produce the KV mask, and a column-wise OR operation is further applied to generate the KV status vector. Importantly, no large intermediate buffer is required for storing the internal spike sequences, because these logic operations can be performed during the write-back of the V spikes while reading the corresponding K spikes. For example, when one row of V spikes is written back, the corresponding row of K spikes can be read out simultaneously, and the AND and OR operations are performed on the fly to update the KV status vector. In the second stage, we compute the attention spikes by performing the AND operation between each row of the Q spikes and the shared KV status vector.

IV. Experimental Results

We implemented and evaluated four spiking models for classification, including VGG11 on CIFAR–10, SpikingFormer-4-256 on CIFAR–10, ResNet18 on CIFAR–10, and SpikingFormer-2-512 on CIFAR–100. For segmentation, we designed a spiking SegNet [21] with the architecture 8C3–16C3–32C3–32C3–16TC3–2TC3, where XCY denotes a convolution layer with X output channels and a kernel size of $Y \times Y$, and TC denotes transposed convolution. All models were trained using AdamW with a batch size of 128. The training was conducted in PyTorch and SpikingJelly [22] using LIF neurons ($\tau = 0.5$) on 4 NVIDIA RTX PRO 6000 GPUs. ExSpike was designed in Verilog HDL, synthesized using Synplify, and implemented in Xilinx Vivado. The final design runs at 200 MHz on an AMD Xilinx Virtex-7 XC7V2000T FPGA.

A. Efficiency of Event Execution with Compression

Fig. 7 evaluates the efficiency of APEC under different group sizes in the VGG11, ResNet18, and SpikingFormer models. In general, APEC consistently improves throughput on all benchmarks, while GROUP=2 (G2) achieves the best result in every case. Compared to baseline, G2 improves average throughput by 10.9%, 13.8%, 14.5%, and 11.2% on VGG11 (CIFAR–10), ResNet18 (CIFAR–10), SpikingFormer-4-256 (CIFAR–10), and SpikingFormer-2-512 (CIFAR–100), respectively. Meanwhile, the corresponding event reduction ratios are 1.62×, 1.35×, 1.36×, and 1.38×, confirming that APEC effectively reduces the number of valid events to be executed and translates this reduction into throughput gains.

The inset plots further reveal that the average O_G decreases rapidly as the group size increases. For example, on SpikingFormer-4-256 (CIFAR–10), the average O_G decreases from 19.08 for G2 to 6.82 for GROUP=4 (G4) and 2.92 for GROUP=8 (G8). A similar trend is observed on all other benchmarks. This directly validates the theoretical analysis in Section III-A2: although a larger group provides a higher theoretical reuse factor, the higher-order overlap shrinks quickly as more adjacent spike sequences are grouped together, which weakens the practical compression gain. As a result, the overall benefit of APEC does not increase monotonically with the group size, and G2 provides the best practical trade-off between the reuse factor and the rapidly diminishing higher-order overlap. Based on these results, G2 is adopted as the default APEC configuration in ExSpike.

Another notable observation is that event reduction does not always translate into proportional throughput improvement for every layer. Some layers, such as Conv6+7 in VGG11 and Enc0 in SpikingFormer, exhibit reduced event

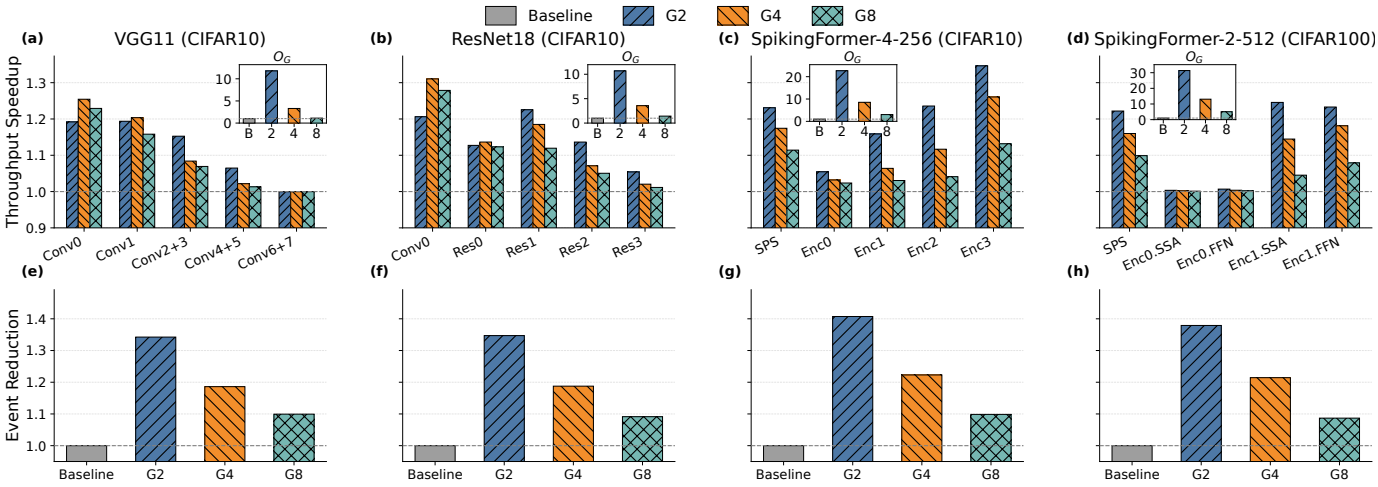


Fig. 7: Performance analysis under different APEC group sizes. (a)–(d) show the block-wise throughput speedup over the baseline without APEC for VGG11, ResNet18, SpikingFormer-4-256, and SpikingFormer-2-512, respectively. VGG11 is divided into five convolutional blocks (Conv$x$), ResNet18 is divided by residual blocks (Res$x$), SpikingFormer-4-256 is divided into the spiking patch splitting (SPS) stage and encoder blocks (Enc$x$), and SpikingFormer-2-512 is partitioned more finely into SPS, spike-driven self-attention (SSA), and feedforward network (FFN) components. (e)–(h) show the corresponding total event reduction compared with the baseline without APEC.

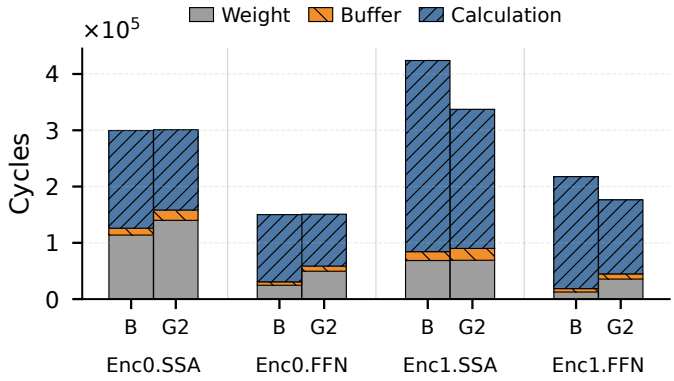


Fig. 8: Latency breakdown for SpikingFormer-2-512 under Baseline and APEC-2. For each block, the total execution latency is decomposed into waiting-for-weight-ready (Weight) cycles, buffer cycles, and calculation cycles.

counts but only marginal throughput changes, suggesting that their performance is constrained by other architectural bottlenecks beyond event-driven accumulation alone. As a concrete example, Fig. 8 shows the cycle-level latency breakdown of SpikingFormer-2-512 before and after applying APEC-2. Although APEC-2 reduces the calculation cycles in both Enc0.SSA and Enc0.FFN, the Weight cycles increase significantly, offsetting the computation reduction and resulting in limited end-to-end latency improvement. Therefore, the benefit of APEC is most pronounced in layers where execution is dominated by event-driven computation and where adjacent spike sequences exhibit strong spatial correlation.

Table I presents the detailed resource breakdown and total power consumption under two architectural configurations, including Baseline and APEC-2. As discussed in Section III-A2, the main overhead of APEC comes from the additional

TABLE I: Resource breakdown of ExSpike. Values are reported as Baseline/APEC-2 where applicable.

	EPE Core	Attention Core	EAFC Core	Sparse Core	Total
kLUTs	19 / 25	6	3	4	36 / 41
kFFs	21 / 26	4	1	2 / 3	36 / 41
BRAMs	128	0	2.5	0	312
Power (W)	1.593 / 1.700				

buffers used to store the partial sums of overlapped sequences. Therefore, when ExSpike is configured with an APEC group size of 2, the resource usage of the EPE Core increases accordingly, with LUT usage rising from 19k to 25k and FF usage increasing from 21k to 26k, while the resources of the other cores remain unchanged. This is because the EPE Core is the main computation engine that supports APEC execution. Meanwhile, the total power consumption increases by 1.07× due to the additional hardware overhead. However, this increase does not offset the performance gain provided by APEC. In particular, for SpikingFormer-4-256 on CIFAR-10, APEC improves throughput by 1.14×, still resulting in an overall energy-efficiency improvement of 1.07×.

B. Comprehensive Evaluation with Prior Work

1) Performance and Energy Efficiency Analysis

Table II compares ExSpike with prior state-of-the-art FPGA-based SNN accelerators in terms of accuracy, throughput, and energy efficiency. For conventional SCNN models on CIFAR-10, ExSpike achieves the best overall balance between accuracy and efficiency. Specifically, for VGG11, ExSpike attains 93.89% accuracy, improving upon NEURAL [20] and SConvNSys [25] by 0.44% and 2.41%, respectively. Meanwhile, it also delivers the highest throughput and energy efficiency among these VGG-style implementations, reaching

TABLE II: Comparison with state-of-the-art FPGA-based SNN accelerators.

Work	Device	Clk (MHz)	Benchmark			Resource					Performance			
			Dataset	Model	Acc.	PE Size	kLUTs	kFFs	BRAM	DSPs	FPS	GOPS	GOPS/W	GOPS/W/PE[1]
Cerebron [21]	xc7z100	200	CIFAR-10	MobileNet	91.90	256	85	70	283	0	90	44.2	31.6	0.12
			MLND_Capstone	SegNet[2]	97.30	256	85	70	283	0	1250	45.0	32.1	0.13
DeepFire2 [23]	xcvu9p	550	CIFAR-10	VGG10	87.10	–	125	–	511	2025	23255	10400	517.93	–
SpikeTA [24]	xcu280	300	CIFAR-100	SpikingFormer-2-512[4]	78.40	35622	503	–	1366	7249	–	28980	405.60	0.01
NEURAL [20]	xc7v2000t	200	CIFAR-10	VGG11	93.45	256	74	63	137.5	0	68	41.37	52.37	0.20
STISA [17]	xczu9eg	200	CIFAR-10	ResNet18	95.29	387	137	137	160	0	49	126.28	74.72	0.19
FireFly-T [18]	xczu5ev	300	CIFAR-10	SpikingFormer-4-256[4]	94.45	8192	46	117	100	304	907	3029	696.64	0.08
			CIFAR-100	SpikingFormer-2-512[4]	78.38	8192	46	117	100	304	630	3317	762.87	0.09
SConvNSys [25]	xcu250	250	CIFAR-10	VGG11	91.48	1024	308	243	324.5	0	73	183.17	105.3	0.10
			MLND_Capstone	USegNet[3]	99.00	800	312	257	287	3	154	43.5	24.2	0.03
ExSpike	xc7v2000t	200	CIFAR-10	VGG11	93.89	352	41	40	312	0	148	361.51	211.91	0.60
			CIFAR-10	ResNet18	94.98	352	41	40	312	0	85	209.31	120.64	0.34
			CIFAR-10	SpikingFormer-4-256[4]	94.45	352	40	40	312	0	197	479.15	281.85	0.80
			CIFAR-100	SpikingFormer-2-512[4]	75.81	352	41	40	335	0	51	463.90	267.53	0.76
			MLND_Capstone	SegNet[2]	98.70	352	43	44	310	0	1633	123.25	82.78	0.24

[1] The normalized efficiency in GOPS/W/PE (GOPS per watt per PE).
[2] SegNet: 8C3–16C3–32C3–32C3–16TC3–2TC3.
[3] UsegNet: 16C3–16C3–64C3–MP2–128C3–MP2–64TC3–16TC3–1TC3.
[4] SpikingFormer-L-D means spiking transformer model with L spiking transformer encoder blocks and D feature embedding dimensions.

148 FPS and 211.91 GOPS/W. For ResNet18 on CIFAR-10, STISA [17] achieves slightly higher accuracy by exploiting temporal optimization to reduce the effective timesteps of different network blocks. Since its reported result is based on a compressed timestep configuration, we set the inference timestep of ExSpike to be consistent with that setting for a fair comparison. Under this condition, ExSpike achieves 1.73× higher throughput and improves energy efficiency by 45.92 GOPS/W, while using a smaller PE size. Although its accuracy is 0.31% lower than STISA, ExSpike achieves substantially higher normalized energy efficiency per PE, indicating better architectural efficiency under constrained resources.

For emerging spiking transformer models, ExSpike also demonstrates competitive deployment capability. On SpikingFormer-4-256 for CIFAR-10, ExSpike achieves 94.45% accuracy, matching FireFly-T [18]. Although FireFly-T reports higher FPS and GOPS/W, its performance strongly relies on DSP-intensive optimization and a much larger PE array, which improves raw throughput but weakens portability to DSP-limited platforms and may complicate ASIC migration. In contrast, ExSpike adopts a DSP-free full-event computing architecture and still achieves the highest normalized efficiency, reaching 0.80 GOPS/W/PE, which is about 10× higher than FireFly-T. A similar trend can be observed on SpikingFormer-2-512 for CIFAR-100. While ExSpike reports lower accuracy than SpikeTA [24] and FireFly-T [18], it delivers much higher normalized energy efficiency per PE. Specifically, ExSpike achieves 0.76 GOPS/W/PE, which is nearly 76× and 8.4× higher than SpikeTA and FireFly-T, respectively. This result suggests that ExSpike is particularly effective in converting irregular event sparsity into hardware efficiency, even when deployed on more complex transformer-style SNNs.

For the segmentation workload, ExSpike also shows strong competitiveness. On MLND_Capstone with SegNet, ExSpike achieves 98.70% accuracy, outperforming Cerebron [21] by 1.40%, while also providing higher throughput and energy efficiency. In terms of normalized efficiency, ExSpike improves GOPS/W/PE by about 1.85×, further demonstrating that the proposed architecture is effective not only for image classification but also for segmentation tasks.

2) Resource Analysis

As summarized in Table II, ExSpike achieves higher hardware efficiency with a compact implementation, requiring only about 40k LUTs and 40k FFs across multiple workloads while supporting diverse SNN models and applications. In contrast, SConvNSys [25] relies on multi-timestep parallel execution, which significantly increases logic cost to more than 300k LUTs and over 240k FFs, limiting its practicality on resource-constrained FPGA platforms. Cerebron [21] improves utilization through additional scheduling and control mechanisms, but this also introduces extra hardware overhead. FireFly-T [18] and SpikeTA [24] further depend on DSPs to boost raw throughput, which increases platform coupling and reduces portability under limited DSP budgets. By comparison, ExSpike adopts a DSP-free full-event computing architecture that performs online event detection and valid-event generation in one cycle, and completes SNN computation mainly through lightweight accumulators and efficient event-driven control.

3) Architecture Flexibility and Generality

At the application level, ExSpike supports not only image classification but also segmentation workloads, as demonstrated by the results on CIFAR-10, CIFAR-100, and MLND_Capstone in Table II. At the model level, ExSpike supports a broad range of mainstream SNN topologies, including fully connected networks, SCNNs, residual networks, and spiking transformer models. This flexibility is enabled by the programmable Instruction SRAM and the unified full-event computing architecture, which together support diverse operators such as fully connected, convolution/transposed convolution, shortcut connection, max/average pooling, and spike-

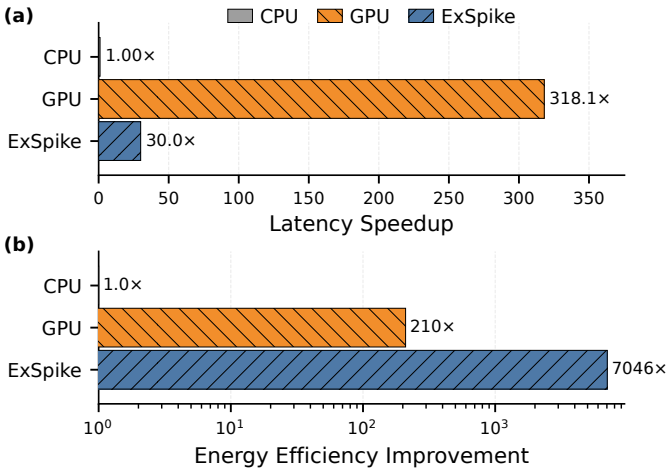


Fig. 9: Performance comparison with Intel(R) Xeon 8470Q CPU and NVIDIA RTX PRO 6000 GPU.

driven self-attention. Therefore, compared with prior FPGA-based SNN accelerators that are often specialized for limited network structures or application scenarios, ExSpike provides a more general and configurable neuromorphic acceleration platform, making it well-suited for exploring practical and diverse SNN deployments.

C. Comparison with CPU and GPU

Fig. 9 compares the performance of ExSpike with an Intel(R) Xeon 8470Q CPU and an NVIDIA RTX PRO 6000 96GB GPU using the SpikingFormer-4-256 classification model on CIFAR-10. Compared with the CPU, ExSpike achieves 30.0× lower latency and 7046× higher energy efficiency, thanks to its specialized full-event architecture for SNNs. Compared with the GPU, ExSpike exhibits higher latency because the GPU executes the model with a larger batch size (32 in this comparison). Nevertheless, ExSpike still delivers nearly 33.6× higher energy efficiency, demonstrating its superior architectural efficiency for SNN inference.

V. Related Work

Event-driven neuromorphic architectures perform computing only after detecting valid spike events, ensuring efficient inference. The spiking multilayer perceptron model (SMLP) is highly suitable for event-driven calculations [26]–[28], since the target region of a single spike is continuous and spans multiple neurons across layers, MP updates can be efficiently performed in parallel and pipelined for higher throughput. However, the representational capacity of SMLP remains limited and can only process simple input patterns.

SCNNs address these limitations by enabling spatially structured, convolution-based spike processing [29]–[31]. Some approaches [14], [21], [23], [25], [32] improve performance by feeding valid spike sequences into computational engines through zero filtering and data reordering under TConv dataflows. Nevertheless, because these architectures are still primarily built on systolic-array- or adder-tree-based computing units [33], [34], they do not fundamentally align execution with sparse spike events, and their processing units continue to suffer from workload imbalance and limited efficiency under irregular sparsity. Other approaches switch to EConv [15], [35], [36], typically by configuring dedicated hardware units for each layer. For direct-coding SCNNs, execution is often divided between separate dense and sparse cores [15], which reduces both resource utilization and computational efficiency. In addition, implementing EConv on general-purpose architectures struggles to maintain the low-power and high-throughput advantages of event-driven computation [36], [37]. SpikeTA [24] and FireFly–T [18] are among the latest architectures delivering high computational performance for spiking transformer models; however, they rely heavily on DSP-based optimizations and do not fully exploit purely event-driven computation, which constrains their per-PE energy efficiency.

As shown in Table III, compared with prior designs, ExSpike targets irregular sparse event dataflow and introduces event compression (EC) under the correctness guarantee of full-event execution, which significantly reduces ineffective accumulations and memory accesses, thereby improving throughput and energy efficiency. Beyond common spiking and convolution operators, ExSpike can support flexible operators such as transposed convolution and spike-driven self-attention calculation. Overall, the architecture maintains generality while fully exploiting irregular sparsity to deliver efficient and scalable acceleration for diverse SNN workloads.

VI. Conclusion

We proposed ExSpike, a full-event neuromorphic architecture developed through dataflow and architecture co-design. By optimizing the key operations that interrupt event-driven execution, ExSpike enabled a pure event-driven dataflow for SNN models. The proposed architecture supported diverse operators, including event-driven convolution and SDSA, while further reducing redundant valid events through APEC. Experimental results demonstrated that ExSpike effectively exploited irregular spike sparsity across diverse SNN workloads, including both image classification and segmentation models. ExSpike also achieved competitive performance with low hardware resource cost, while providing high architectural flexibility and generality for practical neuromorphic deployment.

TABLE III: Feature comparison of representative FPGA-based SNN accelerators.

Feature	SiBrain [14]	STISA [17]	DeepFire2 [23]	SpikeTA [24]	Cerebron [21]	FireFly–T [18]	**ExSpike**
Sparse-aware	✓	✗	✗	✗	✓	✓	✓
Event conv.	✗	✗	✗	✗	✗	✗	✓
Full-event	✗	✗	✗	✗	✗	✗	✓
SDSA	✗	✗	✗	✓	✗	✓	✓
Flex. op.	✓	✓	✓	✓	✓	✓	✓
Dir. coding	✓	✗	✓	✗	✗	✗	✓
DSP-free	✓	✓	✗	✗	✓	✗	✓

Event conv.: Event-driven convolution; SDSA: Spike-driven self-attention computation; Flex. op.: Flexible operator support; Dir. coding: Direct coding support.

References

[1] N. Rathi, I. Chakraborty, A. Kosta, A. Sengupta, A. Ankit, P. Panda, and K. Roy, "Exploring neuromorphic computing based on spiking neural networks: Algorithms to hardware," *ACM Comput. Surv.*, vol. 55, Mar. 2023.

[2] H. Zheng, Y. Wu, L. Deng, Y. Hu, and G. Li, "Going deeper with directly-trained larger spiking neural networks," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, pp. 11062–11070, May 2021.

[3] K. Yu, C. Yu, T. Zhang, X. Zhao, S. Yang, H. Wang, Q. Zhang, and Q. Xu, "Temporal separation with entropy regularization for knowledge distillation in spiking neural networks," in *2025 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8806–8816, 2025.

[4] W. Ye, S. Chen, H. Liu, Y. Liu, Y. Chen, Y. Cui, and W. Lin, "The architecture design and training optimization of spiking neural network with low-latency and high-performance for classification and segmentation," *Neural Networks*, vol. 191, p. 107790, 2025.

[5] H. Zhao, H. Wu, D. Yang, A. Zou, and J. Hong, "Brillm: Brain-inspired large language model," 2025.

[6] P. de Tournemire, D. Nitti, E. Perot, and A. Sironi, "A large scale event-based detection dataset for automotive," *arXiv preprint arXiv*, 2020.

[7] M. Yao, J. Hu, Z. Zhou, L. Yuan, Y. Tian, B. Xu, and G. Li, "Spike-driven transformer," *Advances in neural information processing systems*, vol. 36, pp. 64043–64058, 2023.

[8] C. Zhou, L. Yu, Z. Zhou, H. Zhang, J. Wang, H. Zhou, Z. Ma, and Y. Tian, "Spikingformer: A key foundation model for spiking neural networks," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 40, pp. 2236–2244, 2026.

[9] D. Lee, Y. Li, Y. Kim, S. Xiao, and P. Panda, "Spiking transformer with spatial-temporal attention," in *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13948–13958, 2025.

[10] C. Fang, Z. Shen, Z. Wang, C. Wang, S. Zhao, F. Tian, J. Yang, and M. Sawan, "An energy-efficient unstructured sparsity-aware deep snn accelerator with 3-d computation array," *IEEE Journal of Solid-State Circuits*, vol. 60, no. 3, pp. 977–989, 2025.

[11] R. Yin, Y. Li, A. Moitra, and P. Panda, "Mint: Multiplier-less integer quantization for energy efficient spiking neural networks," in *2024 29th Asia and South Pacific Design Automation Conference (ASP-DAC)*, pp. 830–835, 2024.

[12] C. Wei, B. Duan, C. Guo, J. Zhang, Q. Song, H. Li, and Y. Chen, "Phi: Leveraging pattern-based hierarchical sparsity for high-efficiency spiking neural networks," in *Proceedings of the 52nd Annual International Symposium on Computer Architecture*, ISCA '25, (New York, NY, USA), p. 930–943, Association for Computing Machinery, 2025.

[13] K. Simonyan and A. Zisserman, "Very deep convolutional networks for large-scale image recognition," *arXiv preprint arXiv:1409.1556*, 2014.

[14] Y. Chen, W. Ye, Y. Liu, and H. Zhou, "Sibrain: A sparse spatio-temporal parallel neuromorphic architecture for accelerating spiking convolution neural networks with low latency," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 71, no. 12, pp. 6482–6494, 2024.

[15] I. Aliyev, J. Lopez, and T. Adegbija, "Exploring the sparsity-quantization interplay on a novel hybrid snn event-driven architecture," in *2025 Design, Automation Test in Europe Conference (DATE)*, pp. 1–7, 2025.

[16] T. Li, J. Li, G. Shen, D. Zhao, Q. Zhang, and Y. Zeng, "Firefly-s: Exploiting dual-side sparsity for spiking neural networks acceleration with reconfigurable spatial architecture," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 8, pp. 4007–4020, 2025.

[17] K. Wang, C. Yang, L. Liu, C. Yu, Y. S. Ang, B. Wang, and A. Wang, "Stisa: A 0.16-gops/w/pe single-shot inference fpga-based snn accelerator with algorithm and hardware co-design," *IEEE Transactions on Circuits and Systems I: Regular Papers*, pp. 1–14, 2025.

[18] T. Li, J. Li, G. Shen, D. Zhao, Q. Zhang, and Y. Zeng, "Firefly-t: High-throughput sparsity exploitation for spiking transformer acceleration with dual-engine overlay architecture," *IEEE Transactions on Computers*, pp. 1–14, 2026.

[19] J. Clair, G. Eichler, and L. P. Carloni, "Spikehard: Efficiency-driven neuromorphic hardware for heterogeneous systems-on-chip," *ACM Trans. Embed. Comput. Syst.*, vol. 22, Sept. 2023.

[20] Y. Chen and F. Merchant, "Neural: An elastic neuromorphic architecture with hybrid data-event execution and on-the-fly attention dataflow," in *2026 31st Asia and South Pacific Design Automation Conference (ASP-DAC)*, pp. 1110–1116, 2026.

[21] Q. Chen, C. Gao, and Y. Fu, "Cerebron: A reconfigurable architecture for spatiotemporal sparse spiking neural networks," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, vol. 30, no. 10, pp. 1425–1437, 2022.

[22] W. Fang, Y. Chen, J. Ding, Z. Yu, T. Masquelier, D. Chen, L. Huang, H. Zhou, G. Li, and Y. Tian, "Spikingjelly: An open-source machine learning infrastructure platform for spike-based intelligence," *Science Advances*, vol. 9, no. 40, p. eadi1480, 2023.

[23] M. T. L. Aung, D. Gerlinghoff, C. Qu, L. Yang, T. Huang, R. S. M. Goh, T. Luo, and W.-F. Wong, "Deepfire2: A convolutional spiking neural network accelerator on fpgas," *IEEE Transactions on Computers*, vol. 72, no. 10, pp. 2847–2857, 2023.

[24] Y. Gao, T. Wang, Y. Yang, L. Gong, X. Chen, C. Wang, X. Li, and X. Zhou, "Advancing neuromorphic architecture toward emerging spiking neural network on fpga," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 44, no. 9, pp. 3465–3478, 2025.

[25] W. Ye, Y. Liang, Y. Liu, Y. Cui, Y. Chen, and W. Lin, "Sconvnsys: Accelerating spiking convolutional neural networks with a reconfigurable neuromorphic architecture for diverse applications," *IEEE Transactions on Very Large Scale Integration (VLSI) Systems*, pp. 1–14, 2025.

[26] Y. Liu, Y. Chen, W. Ye, and Y. Gui, "Fpga-nhap: A general fpga-based neuromorphic hardware acceleration platform with high speed and low power," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 69, no. 6, pp. 2553–2566, 2022.

[27] X. Cheng, S. Cao, S. Wang, M. Wang, W. Li, and X. Zeng, "An fpga-based event-driven snn accelerator for dvs applications with structured sparsity and early-stop," *IEEE Transactions on Circuits and Systems I: Regular Papers*, vol. 72, no. 7, pp. 3298–3310, 2025.

[28] A. Carpegna, A. Savino, and S. D. Carlo, "Spiker+: A framework for the generation of efficient spiking neural networks fpga accelerators for inference at the edge," *IEEE Transactions on Emerging Topics in Computing*, vol. 13, no. 3, pp. 784–798, 2025.

[29] Y. Zhang, X. Luo, Q. Sun, Y. Wang, H. Qu, and Z. Yi, "Retina-inspired lightweight spiking convolutional neural network for single-image dehazing," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 36, no. 7, pp. 12580–12594, 2025.

[30] W. Ye, Y. Chen, and Y. Liu, "The implementation and optimization of neuromorphic hardware for supporting spiking neural networks with mlp and cnn topologies," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 2, pp. 448–461, 2023.

[31] S. Panchapakesan, Z. Fang, and J. Li, "Syncnn: Evaluating and accelerating spiking neural networks on fpgas," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, pp. 286–293, 2021.

[32] Y. Chen, Y. Liu, W. Ye, and C.-C. Chang, "The high-performance design of a general spiking convolution computation unit for supporting neuromorphic hardware acceleration," *IEEE Transactions on Circuits and Systems II: Express Briefs*, vol. 70, no. 9, pp. 3634–3638, 2023.

[33] M. T. L. Aung, C. Qu, L. Yang, T. Luo, R. S. M. Goh, and W.-F. Wong, "Deepfire: Acceleration of convolutional spiking neural network on modern field programmable gate arrays," in *2021 31st International Conference on Field-Programmable Logic and Applications (FPL)*, pp. 28–32, 2021.

[34] M. Li, Y. Kan, M. Wu, R. Zhang, and Y. Nakashima, "Fpspike: A fully parallel and reconfigurable architecture for accelerating spiking neural networks with structured sparsity," *IEEE Transactions on Circuits and Systems I: Regular Papers*, pp. 1–14, 2026.

[35] I. Aliyev and T. Adegbija, "Pulse: Parametric hardware units for low-power sparsity-aware convolution engine," in *2024 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, 2024.

[36] Q. Tian, F. Chen, L. Xie, Y. Zhou, Z. Wu, L. Wu, R. Ying, and P. Liu, "Low-cost deployment and acceleration of event-based spiking convolutional neural networks," in *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*, pp. 1–5, 2025.

[37] X. Liu, Z. Pu, P. Qu, W. Zheng, and Y. Zhang, "Activen: A scalable and flexibly-programmable event-driven neuromorphic processor," in *2024 57th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 1122–1137, 2024.